# Factory Control Unit: modellgetriebene Hard- und Software mit 3D-Web-Oberfläche für die Lernfabrik „Factory in a Box“

**Klaus Liebler** · [Hochschule, Fachbereich] · klaus@klaus-liebler.de

## 1 Problem, Kontext und eigener Beitrag

**Problem und Ziel.** Lernfabriken machen Automatisierungstechnik greifbar. Kommerzielle Systeme wie die fischertechnik Training Factory [1] bringen eine festgelegte Steuerung mit, einfache Eigenbauten bieten selten industrieübliche Schnittstellen. „Factory in a Box“ ist eine kompakte Lernfabrik mit Förderband, Roboterarm (Waveshare RoArm-M3), Sensorik und pneumatischer Druckregelstrecke (geplantes Materialbudget rund 600 €). Ziel ist ihre Steuerung, die **Factory Control Unit (FCU)**: eine Platine mit Firmware, die alle Sensoren und Aktoren bedient und über Modbus, OPC UA und eine Browser-Oberfläche zugänglich macht. Das Prozessabbild soll dabei nur einmal beschrieben werden, damit alle Sichten konsistent bleiben.

**Kontext.** OpenPLC [2] bietet eine quelloffene IEC-61131-3-Laufzeit auf Standardhardware. Die FCU ist dagegen ein zweckgebundenes Ein-/Ausgabegerät, dessen Protokollstack und Prozessabbild selbst Lerngegenstand und Erweiterungspunkt sind. Als Schnittstellen dienen Modbus [3] wegen seiner Einfachheit und OPC UA [4, 8] als Bindeglied zur Industrie 4.0.

**Eigener Beitrag.** (a) Eine Steuerplatine auf Basis des STM32H563; (b) ein modellgetriebener Ansatz, bei dem ein Informationsmodell C++-Code für Modbus und OPC UA sowie einen Teil der Web-Oberfläche erzeugt; (c) ein Web-UI mit selbst entwickelter 3D-Engine, digitalem Schatten des Roboterarms und grafischem Teaching; (d) eine Lehranwendung zur Druckregelung; (e) ein Konzept für den Einsatz in vier Lehrveranstaltungen (Kapitel 6).

## 2 Hardware und Firmware

Die Platine vereint einen STM32H563 (Cortex-M33, 640 KiB RAM) mit Ethernet, USB-C (Device, Power-Delivery-Sink) und FDCAN. Für die Anlage sind zwei Schrittmotortreiber (TMC2209, sensorloses Homing), ein Wägezellenwandler, drei Abstands- und ein Farbsensor (I²C), ein Leistungsmonitor (INA226), Lichtschranken, Ventil- und PWM-Treiber, ein Drucksensoreingang, Statusbeleuchtung und ein Bus-Servo-Anschluss für den Roboterarm vorgesehen. Ein EtherCAT-Slave-Controller (LAN9253) ist bestückt und wird derzeit in Betrieb genommen.

Die C++-Firmware läuft auf Eclipse ThreadX [9] mit NetX Duo (TCP/IP, TLS, DHCP, mDNS) und USBX (CDC-ACM für Modbus RTU, CDC-NCM). Ein E/A-Thread bedient im 50-ms-Zyklus alle Sensoren und Aktoren und pflegt das zentrale Registermodell; die Schrittimpulse erzeugt ein Timer-Interrupt. Auf das Registermodell greifen Modbus TCP, Modbus RTU über USB, OPC UA und HTTPS/WebSocket zu. Der OPC-UA-Server ist nach einem Portierungsversuch von open62541 als schlanke Eigenentwicklung (rund 3900 Zeilen C++) entstanden: Basic256Sha256 im Modus Sign, Adressraum als konstante Tabelle im Flash, dasselbe gerätespezifische Zertifikat wie für HTTPS.

## 3 Informationsmodell und Codegenerierung

Das Prozessabbild ist einmalig als annotierte C#-Klassen beschrieben (Abb. 1): 16 Klassen bilden Regionen, 75 Felder Register. Attribute legen Zugriffsrecht, Einheit, Registeradresse, Pin, Wertebereich und Darstellungsart fest; auch Bitfelder und 32-Bit-Werte aus Registerpaaren sind beschreibbar. Ein Generator (UniversalRegisterAccess), vom .NET-Build-Orchestrator aufgerufen, erzeugt daraus drei Artefakte:

(1) **C++-Header für Modbus** mit Adresskonstanten, Regionsgrenzen, Bitmasken und typisierten Accessoren; (2) eine **constexpr-Knotentabelle für OPC UA** (Objekt je Region, Variable je Register mit Datentyp, Zugriffsrechten und Callbacks), die ohne Laufzeitaufbau im Flash liegt; (3) eine **TypeScript-Registerkarte**, aus der das Web-UI seine generischen Register-Panels (Tabellen, Schalter, Schieberegler mit Grenzen) rendert. Schemaänderungen wirken so gleichzeitig auf Firmware und Oberfläche; Inkonsistenzen zwischen den Sichten werden zur Übersetzungszeit vermieden.

Ein zweites Schema (BestBinaryBuffers) beschreibt rund 30 Nachrichten des WebSocket-Protokolls samt Union-Typen für Missionsschritte und Längengrenzen und erzeugt Kodierer, Dekodierer und maximale Puffergrößen für C++ und TypeScript (Tab. 1). Der Orchestrator leitet je Board Hostname und MAC-Adresse aus der Chip-UID ab, erzeugt das TLS-Zertifikat, baut Web-UI (Vite) und Firmware (CMake) und flasht; das Preset Debug-Nucleo erlaubt die Arbeit auf einem Nucleo-H563ZI-Testboard.

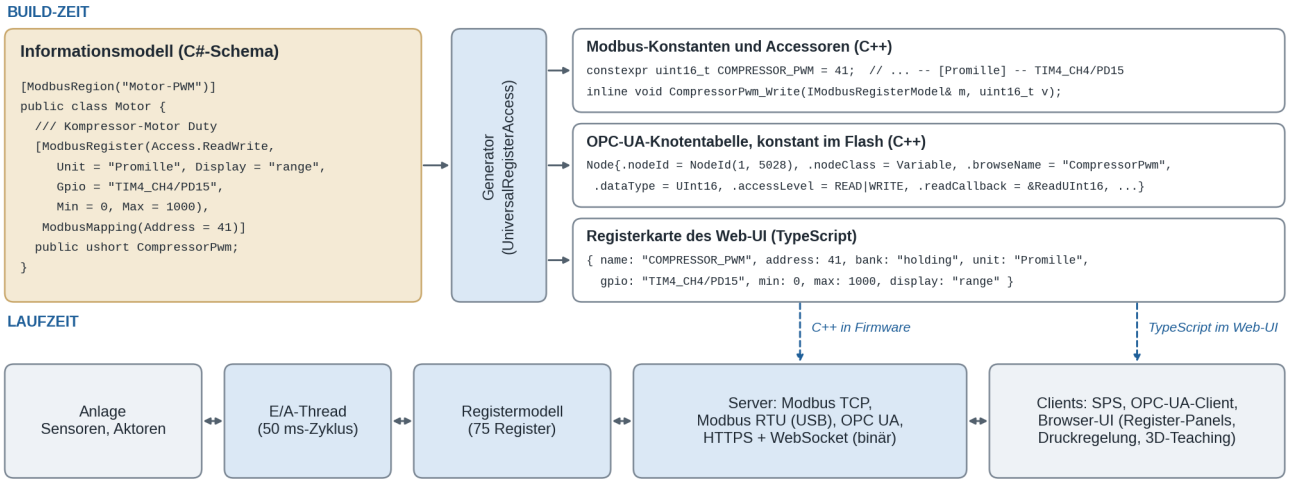



**Abb. 1:** Ein Informationsmodell erzeugt zur Build-Zeit Modbus-, OPC-UA- und Web-UI-Code; zur Laufzeit teilen alle Zugriffswege ein Registermodell.

**Tab. 1:** Aus dem Registerschema (erste drei Zeilen) und dem Nachrichtenschema (letzte Zeile) erzeugte Dateien.

| Artefakt | Sprache | Inhalt | Zeilen |
|---|---|---|---|
| `modbus_registers_generated.hh` | C++ | Adresskonstanten, Bitmasken, typisierte Accessoren | 551 |
| `opcua_registers_generated.hh` | C++ | OPC-UA-Adressraum als constexpr-Tabelle (>110 Knoten) | 902 |
| `register-map.ts` | TS | Registermetadaten für Register-Panels und Regelstrecke | 186 |
| `ws_protocol.hh / ws-protocol.ts` | C++/TS | 30 WebSocket-Nachrichten: Kodierer, Dekodierer, Puffer | 2586 / 1333 |

# 4 Web-Oberfläche

## 4.1 Aufbau und Register-Panels

Das Web-UI ist eine Single-Page-Anwendung aus Lit-Web-Components (TypeScript, Vite; rund 5800 Zeilen in den Anwendungsseiten), als eine Brotli-komprimierte Datei (84 kB statt 460 kB) per HTTPS aus dem Flash der FCU ausgeliefert; alle Anwendungsdaten laufen über einen binären WebSocket. Es umfasst generische Register-Panels, Power Management (INA226), eine Task-Manager-Ansicht mit ThreadX-Live-Statistik, die Roboter-Teach-Seite und die Druckregelstrecke. Die Register-Panels entstehen vollständig aus der generierten Registerkarte, fachliche Seiten finden ihre Register über den Namen in derselben Karte; ein im Schema ergänztes Register erscheint daher ohne UI-Programmierung im Browser.

## 4.2 Druckregelstrecke

Die Druckregelstrecke (Kompressor, Kessel, drei Ventile, Drucksensor) bietet vier Betriebsarten: freies Experiment mit Trenddiagramm; Kennlinie, bei der Studierende je Ventilkombination Punkte aus Kompressorleistung und Maximaldruck von Hand übernehmen; Sprungantwort mit Auswertung von Totzeit und 63-%-Zeitkonstante; Reglerbetrieb (P/PI/PID) mit Entwurf nach T-Summen-Regel oder Chien–Hrones–Reswick [5] und Anti-Windup. Die Firmware sendet alle 500 ms eine kompakte Rückmeldung; der Regelalgorithmus läuft im Browser, die Firmware bleibt ein reines Register-Interface.

# 5 3D-Roboterdarstellung und Teaching

## 5.1 3D-Engine

Auf der schlanken WebGL-Schicht OGL läuft ein eigener PBR-Shader (Cook-Torrance-GGX, analytischer Himmelsverlauf statt Cubemap, daher ohne Bild-Assets). Die Netze des Roboters entstehen offline: Ein Skript liest die URDF-/STL-Daten des Herstellers, verschweißt Vertices und erzeugt ein TypeScript-Modul (272 KB, Abb. 2); Vakuumsauger und Werkstück sind prozedurale Primitive. Vektor- und Quaternionenmathematik, Canvas-Overlay mit Bodenraster und selbst implementiertem Verschiebe-/Drehgizmo samt Hit-Testing (Welt- oder Lokalsystem), SVG-ViewCube und Orbit-/Pan-/Zoom-Kamera sind eigener Code (rund 1500 Zeilen TypeScript).

Eine geschlossene (nicht iterative) inverse Kinematik löst den 3R-Arm mit Basisdrehung: Der Azimut liefert die Basisdrehung, der Handgelenkpunkt über den Kosinussatz die übrigen Winkel; von vier Konfigurationen wird die der aktuellen Pose nächste gewählt (an 2000 Zufallsposen geprüft). Das Raster färbt sich bei Annäherung an Gelenkgrenzen oder bei unerreichbarem Ziel gelb bzw. rot.

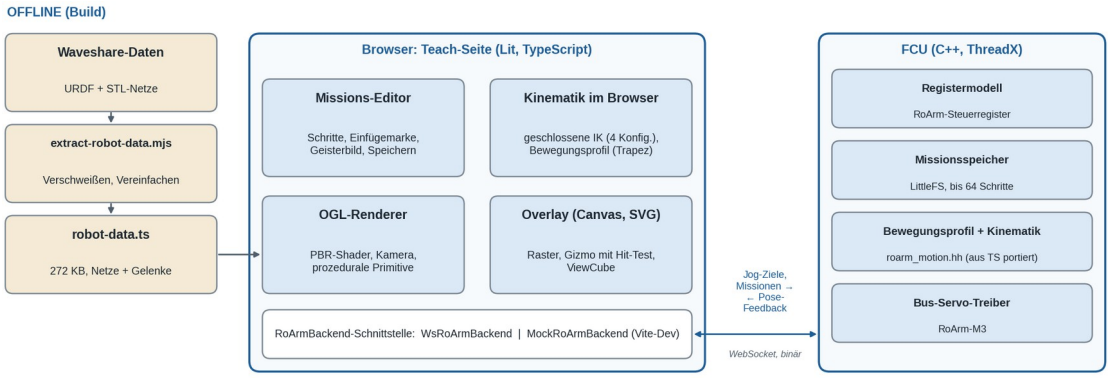



**Abb. 2:** Teach-Seite: Roboterdaten aus der Offline-Werkzeugkette, Browser-Komponenten und Kopplung zur FCU über eine Backend-Schnittstelle.

### 5.2 Teaching und Missionen

Zielposen entstehen über Gelenkschieber, kartesische Werte oder den Gizmo (Abb. 3); die Kinematik läuft im Browser, die FCU erhält Jog-Ziele und meldet Gelenkwinkel, Position und Servostatus zurück, womit das 3D-Modell als digitaler Schatten der realen Pose folgt. Missionen bestehen aus Gelenkfahrten (exakte Pose oder Stützstelle), Vakuum-Schaltschritten und Wartezeiten. Sie werden an einer Einfügemarke bearbeitet (Verschieben, Löschen, Geisterbild der vorherigen Pose), als benannte Slots mit bis zu 64 Schritten im LittleFS der FCU gespeichert und lassen sich per Register auch von Modbus- oder OPC-UA-Clients starten; ein aktiver Teach-Modus sperrt den Missionsstart.

Das trapezförmige Geschwindigkeitsprofil und die Kinematik der Waveshare-Referenz wurden zunächst in TypeScript entwickelt und identisch nach C++ portiert; sie sind noch nicht an realer Hardware verifiziert. Eine Backend-Schnittstelle trennt Oberfläche und Transport: Ein Mock-Backend im Entwicklungsserver erlaubt die UI-Entwicklung ohne Hardware; Abb. 3 stammt aus diesem Aufbau.

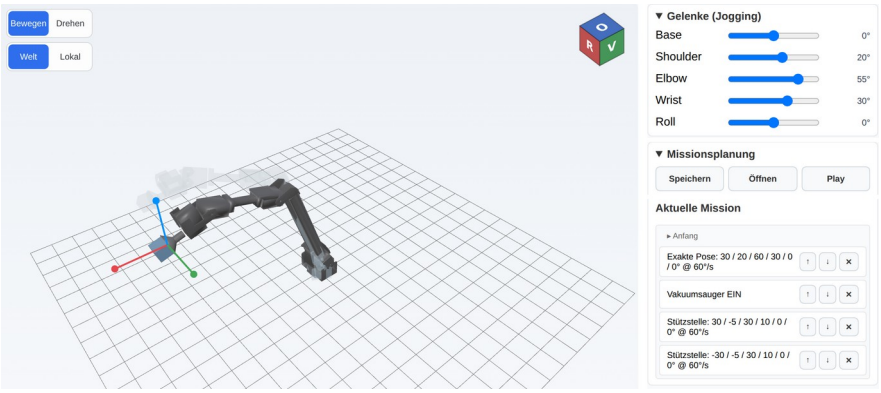



**Abb. 3:** Teach-Seite (Ausschnitte): 3D-Modell mit Gizmo, ViewCube und Geisterbild; Gelenkschieber und Missionsliste (Entwicklungsserver, Mock-Backend).

## 6 Geplanter Einsatz in der Lehre

Die FCU soll in vier Lehrveranstaltungen eingesetzt werden (Tab. 2); der Einsatz ist geplant und noch nicht evaluiert. Alle arbeiten an derselben, greifbaren Anlage, setzen aber an unterschiedlichen Schnittstellen an.

**Tab. 2:** Geplanter Einsatz der FCU in vier Lehrveranstaltungen.

| Veranstaltung | Lerninhalte | Nutzung der FCU und Aufgaben |
|---|---|---|
| **Connected Systems** (Master) | Kommunikationsgrundlagen, Signierung und Verschlüsselung in OPC UA, Echtzeitprotokoll EtherCAT | Modbus und OPC UA (Basic256Sha256, Sign, Gerätezertifikat) auf einem Prozessabbild; EtherCAT-Slave (LAN9253); Mitschnitte und Zertifikate untersuchen |
| **Regelungstechnik** | Streckenidentifikation, Reglerentwurf, Anti-Windup | Druckregelstrecke: Kennlinie, Sprungantwort, Reglerbetrieb (P/PI/PID); Regler einstellen und vergleichen |
| **Automatisierungstechnik** | IEC 61131-3, Ablaufsprache (AS), Funktionsbausteinsprache (FBS) | Fabrik als reale Anlage; Soft-SPS (z. B. CODESYS) über OPC UA oder Modbus; Sortier- und Transportablauf, Inbetriebnahme |
| **Embedded Systems** | Hardwarenaher Entwurf, Echtzeitbetriebssystem, Treiber | Modulare Platine, Testboard Nucleo-H563ZI, reproduzierbarer Build; Baugruppe oder Submodul weiterentwickeln |

### 6.1 Master-Vorlesung „Connected Systems“

Die FCU dient als Anschauungsobjekt für Kommunikation, Absicherung und Echtzeitfähigkeit. Zunächst werden Kommunikationskonzepte (Master/Slave, Client/Server, Registerabbild gegenüber Informationsmodell) an Modbus und OPC UA auf demselben Prozessabbild verglichen. Für OPC UA wird das Sicherheitskonzept praktisch nachvollzogen: Der Server nutzt Basic256Sha256 im Modus Sign mit gerätespezifischem Zertifikat; Studierende untersuchen Zertifikate und Vertrauenskette, stellen im Netzwerkmitschnitt fest, dass signierte Nachrichten manipulationsgeschützt, aber lesbar sind, und diskutieren den Schritt zu Sign and Encrypt [4]. Als Echtzeitprotokoll dient EtherCAT [7]: Der Slave-Controller der Platine wird in einen Verbund aufgenommen, Zykluszeit und Determinismus werden mit Modbus und OPC UA verglichen.

### 6.2 Regelungstechnik: Druckregelstrecke

Die Druckregelstrecke hat eine nichtlineare, über die Ventilkombination veränderliche Kennlinie und deckt den Weg von der Identifikation bis zum Regler ab: Studierende nehmen Kennlinien auf, bestimmen aus Sprungantworten Totzeit und Zeitkonstante, entwerfen P-, PI- und PID-Regler nach Einstellregeln und beobachten im realen Regelkreis Stellgrößenbegrenzung und Anti-Windup. Da der Regelkreis nur über Register geschlossen wird, kann der Regler auch in einem Skript oder einer SPS laufen.

### 6.3 Automatisierungstechnik: Steuerungsentwurf nach IEC 61131-3

Die Fabrik ist der Prozess, für den Studierende die Steuerlogik nach IEC 61131-3 [6] entwickeln, wahlweise in Ablaufsprache (AS) oder Funktionsbausteinsprache (FBS), etwa in CODESYS als Soft-SPS. Die Kommunikation mit der Fabrik erfolgt über OPC UA oder Modbus, sodass beide Wege verglichen werden können. Typische Aufgaben sind ein Sortier- und Transportablauf mit Förderband, Lichtschranken, Ventilen und Sensoren sowie der Aufruf von Roboter-Missionen als Teilablauf; die Register-Panels dienen der Inbetriebnahme.

### 6.4 Embedded Systems: Weiterentwicklung von Baugruppen

Die Platine gliedert sich in funktionale Baugruppen (Antriebe, I²C-Sensorik, Leistungserfassung, Ventil- und PWM-Ausgänge, Kommunikation); Studierende können einzelne Baugruppen oder Submodule weiterentwickeln, etwa einen Sensorzweig, eine Treiberstufe oder eine Erweiterungsplatine samt Firmware-Treiber. Reproduzierbarer Build und Nucleo-H563ZI-Testboard erlauben Arbeit ohne eigene Platine; neue Register stehen nach Ergänzung im Schema in Modbus, OPC UA und Web-UI bereit.

## 7 Reifegrad, Nutzen und Ausblick

Die grundsätzliche Funktion aller Ein-/Ausgänge und Kommunikationsschnittstellen wurde in einer Bachelorarbeit geprüft; EtherCAT wird in Kürze funktionsfähig sein. Codegenerator, Web-UI, 3D-Teaching und Druckregelstrecke sind implementiert (Funktionsmuster im Labor, etwa TRL 4). Offen sind der Einsatz in den genannten Lehrveranstaltungen mit Auswertung von Studierendenfeedback, Zeitmessungen, die Kalibrierung des bislang als Rohwert verarbeiteten Drucksignals und die Verifikation der Roboterkinematik an realer Hardware.

Zum Konferenzthema **Energiemonitoring** liefert der INA226 Spannung, Strom und Leistung der Anlage über alle Schnittstellen; in der Lehre lassen sich Ablaufvarianten energetisch vergleichen. Für **Praxis und Forschung** zeigt der Ansatz, wie ein Informationsmodell konsistente Feldbus-, OPC-UA- und Web-Sichten aus einer Quelle erzeugt und auf einem preiswerten Mikrocontroller Platz findet.

## Literatur


[1] fischertechnik GmbH: Training Factory Industry 4.0 24V – Accompanying booklet, Stand 22.03.2024.

[2] Alves, T. et al.: OpenPLC: An open source alternative to automation. IEEE Global Humanitarian Technology Conference (GHTC), 2014.

[3] Modbus Organization: MODBUS Application Protocol Specification V1.1b3, 2012.

[4] IEC 62541: OPC Unified Architecture, Teil 2 (Sicherheitsmodell) und Teil 6 (Mappings).

[5] Chien, K. L.; Hrones, J. A.; Reswick, J. B.: On the automatic control of generalized passive systems. Trans. ASME 74 (1952), S. 175–185.

[6] IEC 61131-3: Speicherprogrammierbare Steuerungen, Teil 3: Programmiersprachen, 2013.

[7] IEC 61158 (Typ 12): Industrial communication networks – Fieldbus specifications (EtherCAT).

[8] Mahnke, W.; Leitner, S.-H.; Damm, M.: OPC Unified Architecture. Springer, Berlin, Heidelberg 2009.

[9] Eclipse Foundation: Eclipse ThreadX (ehemals Azure RTOS ThreadX). threadx.io